Lecture 2
Inductive Synthesis and Programming by Example

Armando Solar-Lezama

Some Logistics

Join Piazza

- https://piazza.com/mit/spring2015/6885

PSet 1 posted

- http://bit.ly/1Kt1W8F

Optional Office Hour/Recitation this Friday

- Fridays 1-2pm (32-G840)

Next week's meeting

- Tuesday (2/17) and Wednesday (2/18)

Programming by Example

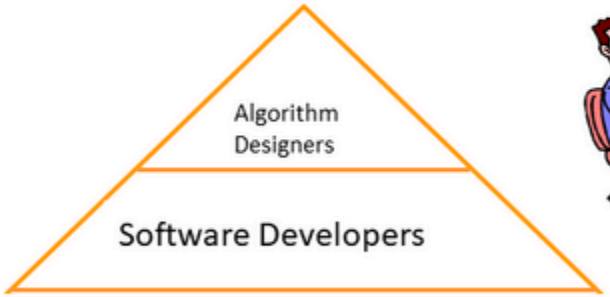
Outline

- Motivation
- History
- Key issues
- Defining the space of programs
- Explicit search

Programming by Example: Motivation

Two major criticisms of synthesis:

- It's too hard to make it work
- Even if it works, it ends up being too hard to use





(logics, automata, etc.)

Most Useful Target



(Examples!)

Programming by Example



PBE vs. PBD

Programming by Example (PBE)

- Generally input/output
- E.g., factorial(6) = 720

Programming by Demonstration (PBD)

- In addition to input/output, show a trace of the computation
- E.g., factorial(6) = 6 * (5 * (4 * (3 * (2 * 1)))) = 720

PBD/PBE vs. Inductive learning

1, 2, 4, 8, ??

1, 2, 4, 7, ??

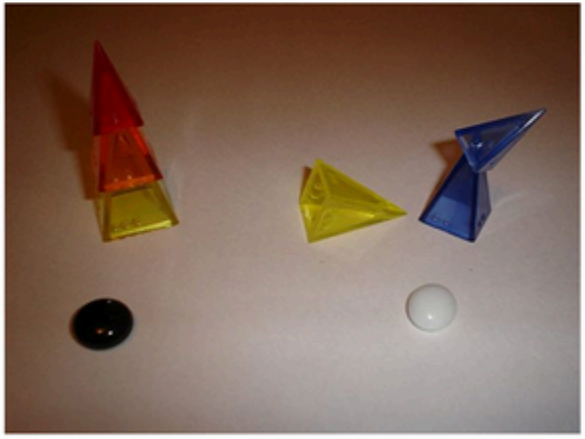
PBE/PBD are examples of inductive learning

Most machine learning falls into this category as well

A little bit of history: who was the first?

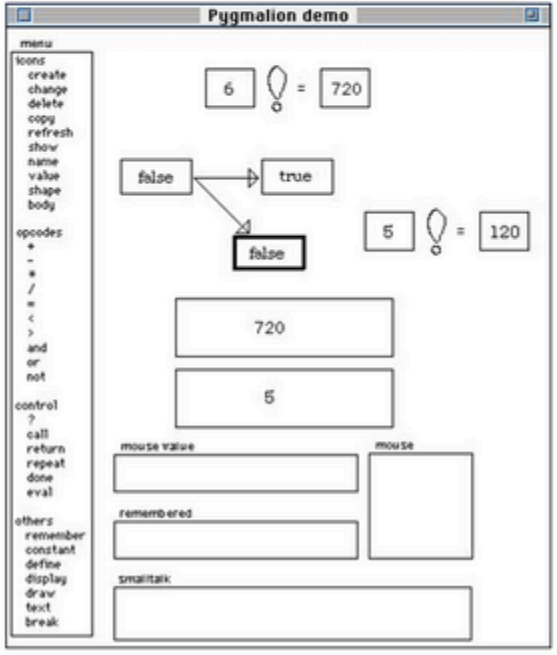
Learning Structural Descriptions from Examples

- By our very own Patrick Winston [1970]
- Explored the question of generalizing from a set of observations
- Similar to the Zendo game:



Pygmalion [Smith 75]

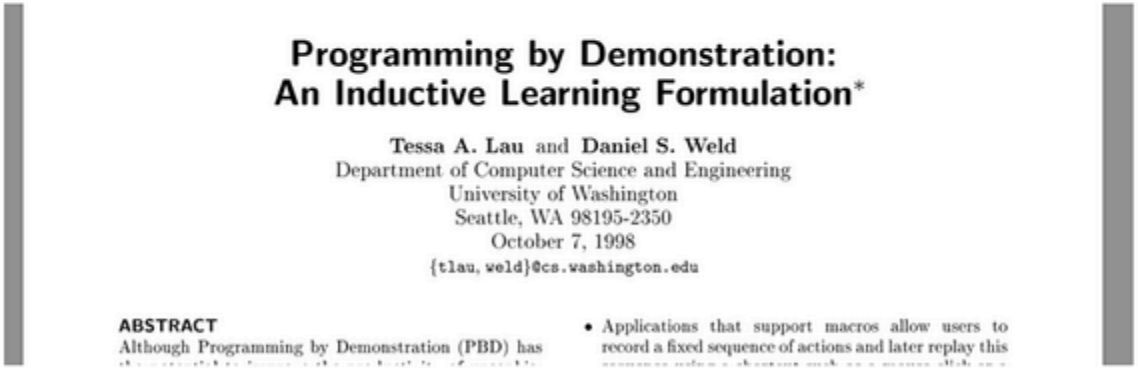
- Real programming by demonstration
- http://acypher.com/wwid/Chapters/01Pygmalion.html



A little bit of history: inductive learning

Ad-hoc approaches → General inductive learning techniques

- Version-space generalization
- Inductive logic programming



Programming by Demonstration: An Inductive Learning Formulation*

Tessa A. Lau and **Daniel S. Weld**
Department of Computer Science and Engineering
University of Washington
Seattle, WA 98195-2350
October 7, 1998
{tlau, weld}@cs.washington.edu

ABSTRACT
Although Programming by Demonstration (PBD) has

• Applications that support macros allow users to record a fixed sequence of actions and later replay this



Tessa Lau

Example

Goto Message #0, whose sender is Corey
Goto Message #1, whose sender is Oren
Goto Message #2, whose sender is Dan
Forward the current message to Tessa
Goto Message #3, whose sender is Corey
Goto Message #4, whose sender is Dan
Forward the current message to Tessa
Goto Message #5, whose sender is Corey
Goto Message #6, whose sender is Corey



Action	Command	Arg	Message	Text	File	Sender	CurrentMesg	PrevCommand
a1	goto	arg1	mesg1			dan	null	null
a2	copy	arg2		subject			mesg1	goto
a3	paste	arg3			file1		mesg1	copy
a4	goto	arg4	mesg2			tessa	mesg1	paste
a5	copy	arg2		subject			mesg2	goto
a6	paste	arg3			file1		mesg2	copy
a7	goto	arg6	mesg3			oren	mesg2	paste
a8	copy	arg2		subject			mesg3	goto
a9	paste	arg3			file1		mesg3	copy



```
command(A,goto) :- prevcommand(A,paste).
command(A,copy) :- prevcommand(A,goto).
command(A,paste) :- prevcommand(A,copy).
command(A,goto) :- prevcommand(A,null).

argument(A,R):-prevcommand(A,goto), textofarg(R,T).
argument(A,R):-prevcommand(A,copy), fileofarg(R,F).
argument(A,R):-currentmesg(A,M), prevcommand(A,
              paste), mesgofarg(R,N), inseq(M,N).
argument(A,R):-currentmesg(A,null),
              mesgofarg(R,mesg1).
```



```
For each message m in Mailbox:
     Goto(m)
     If Sender(m) is Dan:
          Then Forward(m, Tessa)
```

A little bit of history

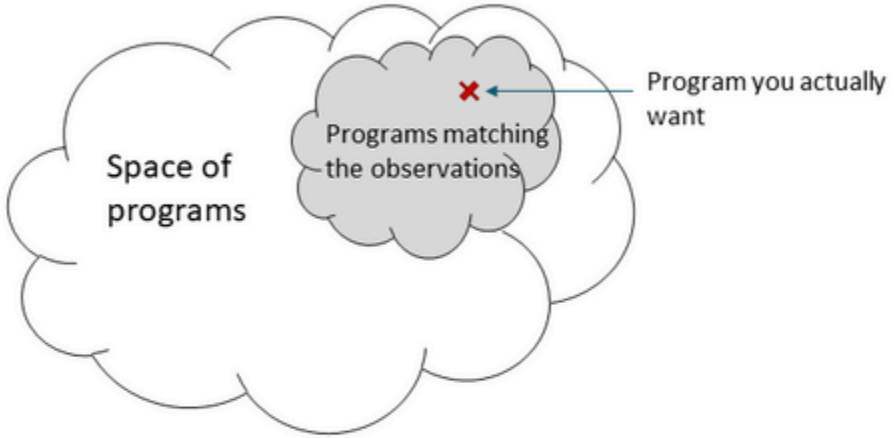
- Detect failure and fail gracefully
- Make it easy to correct the system
- Encourage trust by presenting a model users can understand
- Enable partial automation

Why PBD systems fail: Lessons learned for usable AI

Tessa Lau
IBM Almaden Research Center
650 Harry Road
San Jose, CA 95120 USA
tessalau@us.ibm.com

Abstract
Programming by demonstration systems have long attempted to make it possible for people to program computers without writing code. However, while these systems have resulted in many publications in AI venues, none of the technologies have yet achieved widespread adoption. Usability remains a critical barrier to their success. Based on lessons learned from three different programming by demonstration systems, we present a a set of guidelines to consider when designing usable AI-based systems.

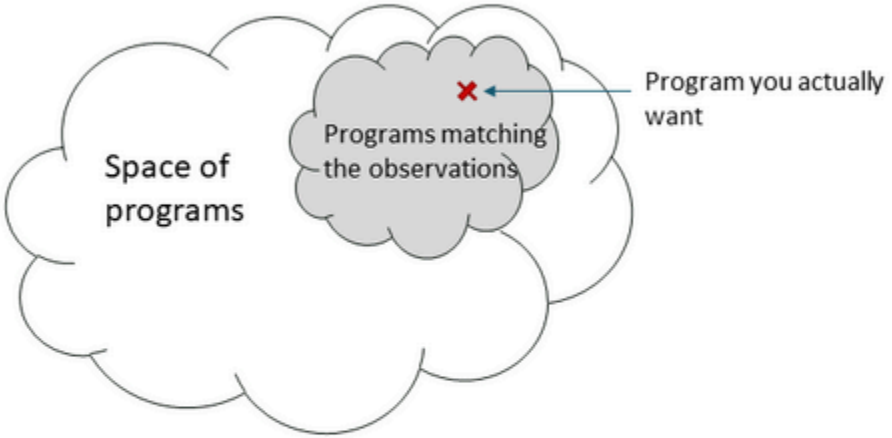
Key issues in inductive learning



(1) How do you find a program that matches the observations?

(2) How do you know it is the program you are looking for?

Key issues in inductive learning



Traditional ML emphasizes (2)

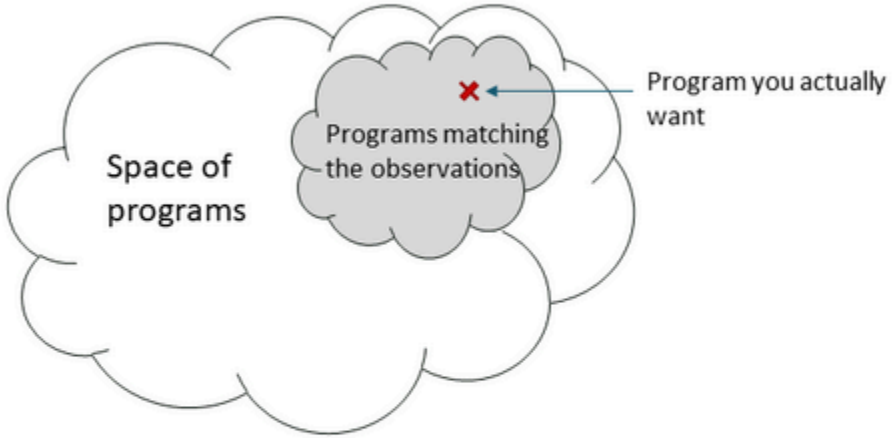
- Fix the space so that (1) is easy

So did a lot of PBD work

(1) How do you find a program that matches the observations?

(2) How do you know it is the program you are looking for?

The synthesis approach

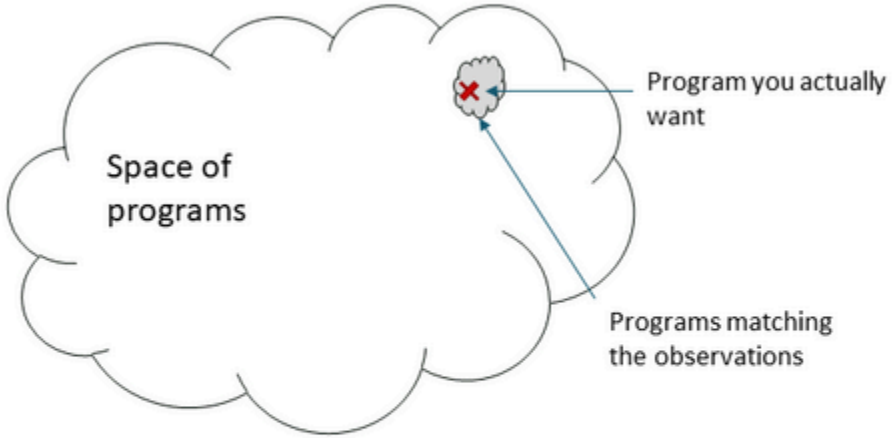


Modern emphasis

(1) How do you find a program that matches the observations?

(2) How do you know it is the program you are looking for?

The synthesis approach



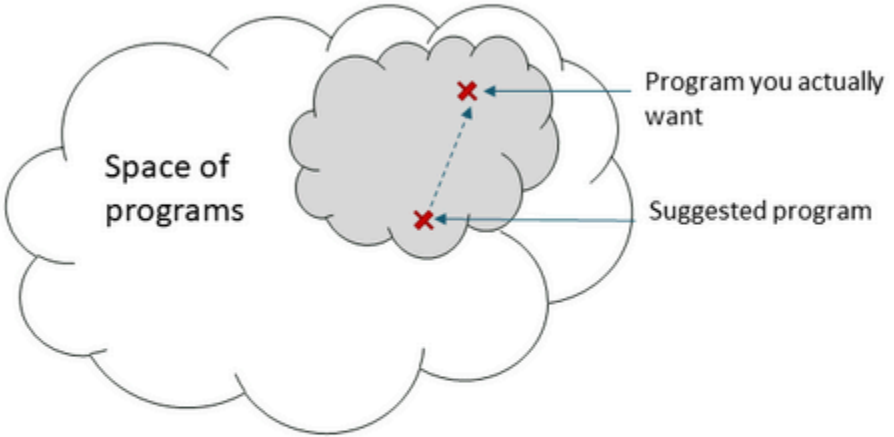
Modern emphasis

- If you can do really well with (1) you can win
- (2) is still important

(1) How do you find a program that matches the observations?

(2) How do you know it is the program you are looking for?

Interaction model

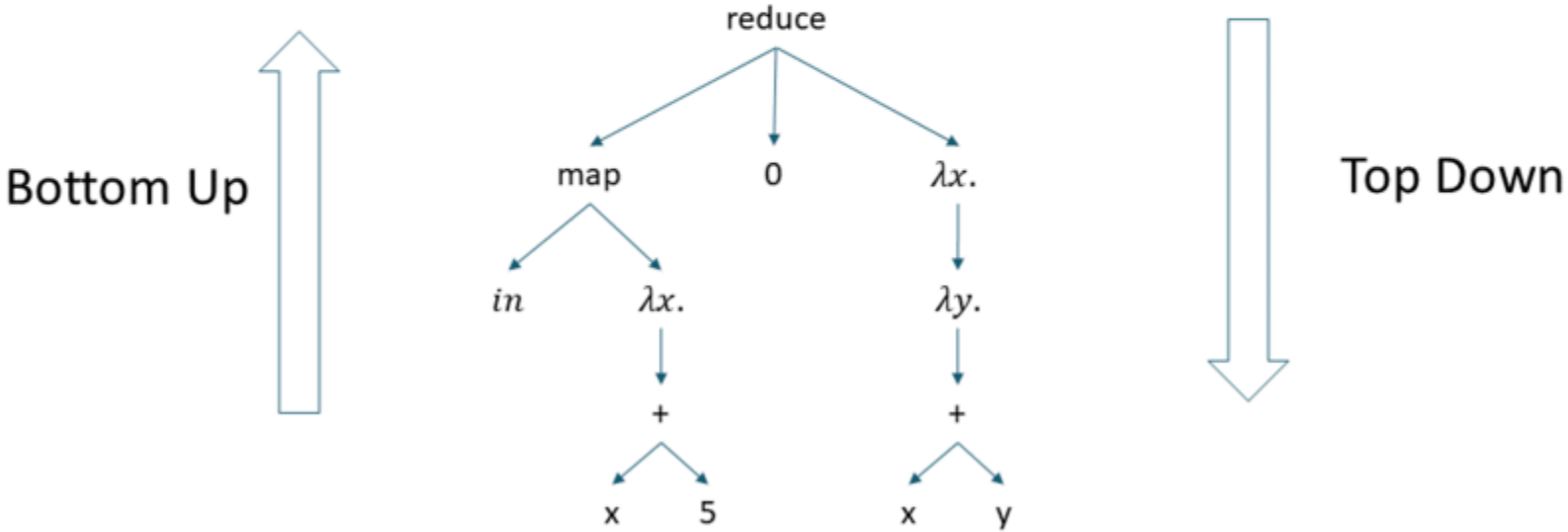


A third issue:

- Historically an HCI topic
- Related to two major questions
  - How to define the search space
  - How to search

Top down vs. Bottom Up Enumeration

reduce (map in λ x. x + 5) 0 $\lambda\, x.\, \lambda y.\, x + y$



Defining the space of programs

Structural hypothesis

- What is the space
  - How do you describe it (user's perspective)
  - How do you represent it (system's perspective)
- Does it have any properties that can help the search

Example

1,4,2,0,7,9,2,5,0,3,2,4,7 → 1,2,4,0,2,5,7,9,0,2,3,4,7,0

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

```
Process(in) := sort(lstExpr[0, firstZero(in)]) + [0] + recursive(lstExpr[firstZero(in)+1, len(in)]);
```

What is the space?

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

The set of all programs in lstExpr

What is the space?

Grammars as definitions of program spaces

- Pro
  - Clean declarative description
  - Easy to sample
  - Easy to explore exhaustively
- Con
  - Insufficiently expressive

Program spaces with grammars

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

What if we know the following:

- Sort is never called more than once in a sub-list.
- Recursive calls should be made on lists whose length is less than len(in)
- We'll never have to add one multiple times in a row

Generators/Generative models

Programs that produce programs

- Can be either random or non-deterministic

Pros:

- Extremely general
  - easy to enforce arbitrary constraints

Cons:

- Extremely general
  - Hard to analyze and reason about
  - Hard to automatically discover structure of the space

Symmetries

Multiple ways of representing the same problem

```
Expr := var*const
      | Expr + Expr
```

```
Expr := var*const
      | var*const + Expr
```

w*c1+(x*c2+(y*c3+z*c4))

- Grammar on the right has fewer symmetries
- Grammar on the left can produce all possible ways to parenthesize
- Can completely eliminate symmetries from the right by enforcing a variable ordering
  - Can't be done with a grammar, but it can with a generative model

```
Expr(vmin) := let v = var() in v*const  (assert v>vmin)
            | let v=var() in v*const + Expr(v) (assert v > vmin)
```

Symmetries

Do symmetries matter?

- It depends

Some methods are very sensitive to symmetries

- E.g. symbolic search

Others are largely oblivious to them

- E.g. sampling

Representing the space

As the system searches, it needs to keep track of what has already been generated and what needs to be generated next

Representation and search strategy are intimately tied together

Explicit Search

Idea

Generate programs one by one

- Generate & Test approach

Key issues

- In what order do you generate?
  - Influences performance *and* result quality
- How do you prune?
  - Essential for scalability
- How do you keep track of the remaining space?
  - Especially challenging in the context of pruning

Explicit search from grammars

Grammar describes how to generate program fragments
from smaller program fragments

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

```
plist := set of all terminals
while(true){
   plist := grow(plist);
   forall( p in plist)
        if(isCorrect(p)){ return p; }
}

Grow(plist){
 // return a list of all trees generated by
 // taking a non-terminal and adding
 // nodes in plist as children
}
```

Explicit search from grammars

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

Grammar describes how to generate program fragments from smaller program fragments

Level 0

in [0] 0



Level 1

in	[0]	0	sort(in)	sort([0])	in[0,0]
[0][0,0]	in + in	in + [0]	[0] + [0]	[0] + in	rec(in)
rec([0])	firstZero(in)	firstZero([0])	len(in)	len([0])	0+1

Set grows very fast!

Large equivalence classes of equivalent programs

Identifying equivalent programs

Program equivalence is hard

- It is also unnecessary!

Observational Equivalence

- Are they equivalent wrt the inputs
  - easy to check efficiently
  - sufficient for the purpose of PBE
- Keep only the simplest one

```
plist := set of all terminals
while(true){
   plist := grow(plist);
   plist := elimEquvalents(plist);
   forall( p in plist)
        if(isCorrect(p)){ return p; }
}
```

Explicit search from grammars

Features:

- Search small programs before large programs
- Simple
- Works even with black-box language building blocks
  - no need to have source for sort or firstZero
    just need to be able to execute them
  - no need to know of any properties about them
    e.g. automatically ignores sort(sort(in)) without
    having to know that sort is idempotent
- Complexity depends on the size of the set of distinct programs
  - Copes well with symmetries

Explicit search from grammars

Limitations:

- Only scales to very small programs
- Unsuitable for programs with unknown constants
  - A single unknown 32-bit constant makes the problem intractable
- Hard to generalize to arbitrary generators
  - Relies heavily on recursive structure of grammar
- Hard to take advantage of additional domain knowledge

Example system:

- Recursive Program Synthesis [Albarghouthi et al., CAV 2013]